

RxNow

Test Plan

Draft v0.2 — for team review

Field	Value
Prepared by	Josh Palmer (QA, CM/DM)
Status	Draft — for internal team review
Tracks	RxNow SRS and openapi.yaml (current main)
Related	SRS, SDD, API-README.md, openapi.yaml, Feature Planning Report

1. How to read this document

This plan is organized so you don't have to read all of it. Start with the role summary on this page; then read the section that names you. The test catalog and shared procedures are there for reference when you need them, not for cover-to-cover reading.

Quick map

Section	Contents
§2	Roles at a glance — one-table view of who owns what.
§3	Your section — pick the one with your name. Self-contained.
§4	Test catalog — every planned test case, tagged with owner.
§5	Shared procedures — environments, defects, traceability.
§6	Notes — open SRS discrepancy, risks, assumptions.

What's changed since v0.1

- Reorganized around ownership rather than IEEE 829 structure.
- Each test case is now tagged with a primary owner so there's no ambiguity about who runs it.
- Updated for Parker's departure — Xander now covers Product Owner, Scrum Master, and Front-End Tech Lead.
- Added a callout in §6 about the workload concentration this creates.

2. Roles at a glance

Each team member is the primary owner of a specific testing slice. "Primary owner" means: you write the cases, run them, and respond when they fail. Others may contribute (e.g., the Database lead provides fixtures the QA lead uses), but the primary owner is accountable.

Person	Role(s)	Owns these testing activities
Josh Palmer	QA / CM / DM	System and regression tests, performance measurement, usability inspection, traceability matrix, defect triage, release gatekeeping.
Xander Weibel	Product Owner, Scrum Master, Front-End Tech Lead	Front-end unit tests, front-end integration tests against Prism then real back-end, acceptance demos against the SRS, sprint planning.
Joe Tolley	Back-End Tech Lead	Back-end unit tests (validation, password hashing, supply calculator, notification engine), API contract conformance, back-end integration with the database.
Haeji Na	Database Tech Lead	Schema constraint tests (DB1–DB9), foreign-key enforcement tests, test fixture seeds, data-layer defect fixes.

Owner tags used in this document

Throughout §4 (the test catalog), each case carries one of the owner tags below. Where a case is mirrored at multiple levels (e.g., a system test has a parallel unit test in the relevant tier), the catalog notes that in the description and the per-role section spells out what the unit-level version looks like.

Tag	Meaning
Josh	Runs at system or regression level. End-to-end through the running application.
Xander (FE)	Runs at front-end unit or front-end integration level.
Joe (BE)	Runs at back-end unit or back-end integration level. Includes API contract conformance.
Haeji (DB)	Runs at the database/schema level.

3. Your section

3.1 Josh (QA / CM / DM)

You own the verification spine. You decide when the product is ready to ship and you produce the evidence that supports that decision.

Your test cases

You run every case in §4 with the owner tag **Josh**. These are the system-level end-to-end cases plus the performance and usability cases:

- **Auth flows:** TC-AUTH-01 through TC-AUTH-12 (end-to-end through the UI or API client).
- **Medication CRUD:** TC-MED-01 through TC-MED-14 (end-to-end).
- **Supply display:** TC-CALC-06.
- **Dashboard:** TC-DASH-01 through TC-DASH-04.
- **Notifications:** TC-NOTIF-05 through TC-NOTIF-08 (display side).
- **Refill workflow:** TC-REFILL-01 through TC-REFILL-11.
- **Usability:** TC-USE-01 through TC-USE-04.
- **Interface:** TC-IF-01 through TC-IF-05.
- **Constraints/attributes:** TC-CON-01 through TC-CON-03, TC-SA-01 through TC-SA-03.
- **Performance:** TC-PERF-01 through TC-PERF-04 (you execute; Joe provides instrumentation if needed).

Your standing responsibilities

- Keep the traceability matrix in §5.3 current — when a case is added, executed, or a defect found, the matrix gets updated the same day.
- Triage new defects within one working day (see §5.2 for severity definitions).
- Run the full regression suite before any release tag.
- Confirm exit criteria (§5.4) are met before approving a release.
- Maintain the test plan itself — when scope or risks change, this document changes.

Your entry criteria for testing a build

- Build merged to main, CI green.
- openapi.yaml lints clean against Redocly.
- Required environment is up — Prism for FE-only cases, real back-end + DB for integration and system cases.
- Appropriate test fixture loaded (empty/standard/performance, per the case being run).
- No blocking defects open from the previous run.

3.2 Xander (PO / SM / Front-End Tech Lead)

You wear three hats this leg. The front-end testing responsibilities below were Parker's; you've absorbed them. See §6.2 for why this is flagged as a risk.

Your test cases (Front-End hat)

Front-end unit tests for the cases below. These mirror system-level cases that Josh runs end-to-end, but yours run in isolation against the FE codebase, with Prism standing in for the back-end where needed.

- **Form validation:** Client-side validation for TC-AUTH-02 (malformed email), TC-AUTH-04 (password policy), TC-MED-02 through TC-MED-07 (medication field constraints). The FE should reject these before submission.
- **AuthController:** Unit tests for the controller logic — processSignup, processLogin, processReset, processLogout — using the names from the Feature Planning Report.
- **AuthService (back interface):** Unit tests confirming the FE-to-API calls match the openapi.yaml shapes. Use openapi-typescript-generated types so drift is a compile error.
- **Component rendering:** Unit tests for AuthComponent render methods (renderSignupForm, renderLoginForm, renderResetForm, destroySessionUI).
- **Dashboard rendering:** Empty-state, single-medication, multi-medication, and low-supply-indicator rendering (supports TC-DASH-01 through TC-DASH-04, TC-USE-03).

Your integration tests (Front-End hat)

- Run the FE against Prism (the mock derived from openapi.yaml) and confirm every screen functions. This can begin immediately — no back-end dependency.
- Once the back-end is up, repoint the FE at it and re-run the same flows. Any divergence is an integration defect.

Your responsibilities (PO / SM hats)

- **Acceptance:** Sign off on demos against SRS acceptance criteria. Josh's system pass + your acceptance demo = ready for release.
- **Priority:** Set defect priority during weekly triage (Josh sets severity, you set priority).
- **Requirement clarifications:** Resolve ambiguity when QA or a tech lead surfaces it — including the SRS discrepancy flagged in §6.1.
- **Scheduling:** Sprint planning and milestone tracking (the milestones from this plan are in §5.5).

3.3 Joe (Back-End Tech Lead)

You own back-end correctness and API contract conformance. The openapi.yaml file is the contract; your tests verify the back-end honors it.

Your test cases

Back-end unit tests covering the business logic and validators. These mostly run with the database mocked or in a transactional sandbox.

- **Password policy enforcement:** Unit tests for the validator behind TC-AUTH-04 — one case per rule (length, uppercase, lowercase, digit).
- **Password hashing:** Unit test confirming the hashing function is one-way, salted, and never returns plaintext (supports TC-AUTH-13, SA3).
- **JWT generation and validation:** Unit tests that issued tokens validate, expired tokens reject, and malformed tokens reject (supports TC-AUTH-14, TC-AUTH-15).
- **Email validation:** Unit tests for the email-format validator (supports TC-AUTH-02).
- **Field validation:** Unit tests for medication field validators — empty name, oversized name, oversized dosage, negative pills_remaining, zero pills_per_day (supports TC-MED-02 through TC-MED-07).
- **Supply calculator:** Unit tests for TC-CALC-01 through TC-CALC-05 — the floor-division logic for days_remaining. Pure function, easy to cover exhaustively.
- **Notification engine:** Unit tests for TC-NOTIF-01 through TC-NOTIF-04 — the threshold-crossing logic, including the anti-spam rule in FR15. This is the trickiest piece of business logic in the system; heavy coverage here pays off.
- **Refill state machine:** Unit tests for the Pending → Sent transition and the rejection of double-sends (supports TC-REFILL-06, TC-REFILL-07).
- **Structured message generation:** Unit test for the refill message format (supports TC-REFILL-05).

Your API contract conformance

- Run an API test runner (Postman, Bruno, or a script in your test framework) against Prism. Every endpoint defined in openapi.yaml gets a happy-path call plus one call per documented error response. Prism validates that responses match the spec.
- Once the back-end is implemented, re-run the same runner against it. Any difference between Prism behavior and real-back-end behavior is a contract violation and gets a defect.
- When openapi.yaml changes (and the workflow in API-README.md says spec changes first, then code), re-run the conformance suite.

Your integration tests

- Back-end ↔ database integration: confirm the data access layer correctly persists, retrieves, and constrains the entities defined in SDD §6.
- Coordinate with Haeji on schema-level cases (§3.4) — your integration tests should rely on her constraint tests passing rather than re-verifying schema rules.

3.4 Haeji (Database Tech Lead)

You own the data layer's integrity. Schema constraints are the last line of defense against bad data. They run before application code gets a chance to misbehave.

Your test cases

Schema-level tests that run directly against the database, bypassing the application:

- **TC-DB-01:** User table has a unique constraint on email and stores password_hash (not plaintext).
- **TC-DB-02:** Medication table has the required primary key and columns.
- **TC-DB-03:** Foreign key Medication.user_id → User.user_id is enforced (an insert with an invalid user_id is rejected).
- **TC-DB-04:** pills_remaining = -1 is rejected by a CHECK constraint or equivalent.
- **TC-DB-05:** pills_per_day = 0 is rejected.
- **TC-DB-06:** Medication name of length 0 or 101 is rejected.
- **TC-DB-07:** Dosage of length 0 or 21 is rejected.
- **TC-DB-08:** No plaintext password is present anywhere in the database (direct query).
- **TC-DB-09:** Refill status column or table exists with the expected constraint (Pending or Sent only).
- **TC-MED-15:** FK enforcement at the data layer (paired with Joe's integration tests).

Your standing responsibilities

- **Test fixtures:** Maintain the three seed sets defined in §5.1. Empty seed, standard seed (3 users, varied medication states), and performance seed (50 users).
- **Schema versioning:** When the schema changes, your constraint tests change with it.
- **Data-layer defects:** Anything the application surfaces that turns out to be a schema or constraint issue is yours to fix.

Your fixture file location

To be decided — pick a location in the repo, document it here, and commit the seeds. Suggestion: db/fixtures/ with one SQL file per seed set.

4. Test catalog

Every planned test case, organized by feature. Each case has an owner tag. Where the same logical behavior is tested at multiple levels (system + unit), this catalog lists the system-level case; the parallel unit tests are described in the relevant role section in §3.

4.1 PF1 — Authentication

ID	Description	Expected Result	Req(s)	Method(s)	Owner
TC-AUTH-01	Register new account with valid email and policy-compliant password	201 with user and JWT	FR1, DB8	Demo, Insp	Josh
TC-AUTH-02	Register with malformed email	400 VALIDATION_FAILED, email detail	FR1	Demo	Josh
TC-AUTH-03	Register with duplicate email	409 EMAIL_TAKEN	FR1	Demo	Josh
TC-AUTH-04	Register with password missing each rule (length/upper/lower/digit), one case per rule	400 VALIDATION_FAILED, password detail	FR2, DB8	Demo, Anal	Josh
TC-AUTH-05	Login with valid credentials	200 with user and JWT	FR3	Demo	Josh
TC-AUTH-06	Login with wrong password	401 INVALID_CREDENTIALS	FR3	Demo	Josh
TC-AUTH-07	Login with unregistered email	401 INVALID_CREDENTIALS (same code as wrong password, to avoid user enumeration)	FR3, SA3	Demo, Insp	Josh
TC-AUTH-08	Logout while authenticated	204; session ends	FR4	Demo	Josh
TC-AUTH-09	Password reset request for registered email	204; reset artifact captured	FR5	Demo, Sim	Josh
TC-AUTH-10	Password reset request for unregistered email	204 (no info leak)	FR5	Demo, Insp	Josh
TC-AUTH-11	Confirm reset with valid token	204; old password no longer works; new one does	FR5	Demo	Josh
TC-AUTH-12	Confirm reset with expired or reused token	410 TOKEN_EXPIRED	FR5	Demo	Josh
TC-AUTH-13	Inspect stored password	Hashed, no plaintext present	SA3, DB1	Insp	Haeji
TC-AUTH-14	Call authenticated endpoint with no token	401 UNAUTHORIZED	FR3, DC3	Demo	Joe
TC-AUTH-15	Call authenticated endpoint with malformed/expired token	401 UNAUTHORIZED	FR3	Demo	Joe

4.2 PF2 — Medication Creation, Editing, Deletion

ID	Description	Expected Result	Req(s)	Method(s)	Owner
TC-MED-01	Create medication with valid fields	201 with computed Medication	FR6, FR7	Demo	Josh
TC-MED-02	Empty name	400 VALIDATION_FAILED	FR6, DB6	Demo	Josh
TC-MED-03	Name longer than 100 chars	400 VALIDATION_FAILED	DB6	Demo	Josh
TC-MED-04	Dosage longer than 20 chars	400 VALIDATION_FAILED	DB7	Demo	Josh
TC-MED-05	pills_remaining = -1	400 VALIDATION_FAILED	DB4	Demo	Josh
TC-MED-06	pills_remaining = 0	201; days_remaining = 0; low_supply = true	DB4, FR10	Demo, Anal	Josh
TC-MED-07	pills_per_day = 0	400 VALIDATION_FAILED	DB5	Demo	Josh
TC-MED-08	List medications for authenticated user	200 with array, scoped to caller	FR12	Demo	Josh
TC-MED-09	Get medication owned by caller	200 with Medication	FR12	Demo	Josh
TC-MED-10	Get medication owned by another user	404 NOT_FOUND (no enumeration)	SA3, DB3	Demo, Insp	Josh
TC-MED-11	Update medication with valid values	200; updated_at advanced	FR8	Demo	Josh
TC-MED-12	Update non-existent medication	404 NOT_FOUND	FR8	Demo	Josh
TC-MED-13	Delete existing medication	204; subsequent GET 404	FR9	Demo	Josh
TC-MED-14	Delete medication owned by another user	404 NOT_FOUND	FR9, DB3	Demo	Josh
TC-MED-15	FK enforcement at DB layer	Invalid user_id insert rejected	DB3	Insp, Anal	Haeji

4.3 PF3 — Supply Calculation

ID	Description	Expected Result	Req(s)	Method(s)	Owner
TC-CALC-01	30 pills, 1/day	days_remaining = 30	FR10	Anal	Joe
TC-CALC-02	30 pills, 2/day	days_remaining = 15	FR10	Anal	Joe
TC-CALC-03	31 pills, 2/day (non-divisible)	days_remaining = 15 (floor)	FR10	Anal	Joe
TC-CALC-04	0 pills, 1/day	days_remaining = 0	FR10	Anal	Joe
TC-CALC-05	1 pill, 1/day	days_remaining = 1	FR10	Anal	Joe
TC-CALC-06	Dashboard days_remaining matches API value	Values agree	FR11	Demo, Insp	Josh

4.4 PF4 — Dashboard

ID	Description	Expected Result	Req(s)	Method(s)	Owner
TC-DASH-01	User with multiple medications logs in	All medications shown in one view	FR12, IR3	Demo	Josh
TC-DASH-02	Inspect each dashboard row	Name, dosage, pills_remaining, days_remaining, low-supply status present	FR13	Demo, Insp	Josh
TC-DASH-03	User with no medications logs in	Empty state, no error	FR12, SA2	Demo	Josh
TC-DASH-04	User with 25 medications logs in	All visible, no pagination errors	FR12, PR4	Demo, Sim	Josh

4.5 PF5 — Low-Supply Notifications

ID	Description	Expected Result	Req(s)	Method(s)	Owner
TC-NOTIF-01	days_remaining drops 8 → 7	One notification at threshold 7	FR14, FR15	Sim, Anal	Joe
TC-NOTIF-02	days_remaining stays at 7 across updates	No additional notification (anti-spam)	FR15	Sim, Anal	Joe
TC-NOTIF-03	days_remaining rises above 7, then back to 7	Second threshold-7 notification fires	FR15	Sim, Anal	Joe
TC-NOTIF-04	days_remaining crosses 7, 3, 1 over time	Three distinct notifications	FR15	Sim, Anal	Joe
TC-NOTIF-05	View notification list while authenticated	All caller's notifications returned	FR16	Demo	Josh
TC-NOTIF-06	Mark notification as read	200; unread_only excludes it	FR16	Demo	Josh
TC-NOTIF-07	Notification message inspection	Plain English, identifies medication and threshold	FR16, UR4	Insp	Josh
TC-NOTIF-08	Low-supply on dashboard for ≤7 day medication	Color indicator AND text label present	UR3	Demo, Insp	Josh

4.6 PF6 — Refill Workflow

Note: SRS §3.1 numbers these FR17–FR23 differently than the verification mapping in §4.6. The cases below track §3.1. See §6.1 for the discrepancy.

ID	Description	Expected Result	Req(s)	Method(s)	Owner
TC-REFILL-01	Initiate refill for owned medication	201; status = Pending	FR17, FR22	Demo	Josh
TC-REFILL-02	Initiate refill for non-existent/non-owned medication	404	FR17, DB3	Demo	Josh
TC-REFILL-03	Create provider via POST /providers	201 with Provider	FR18	Demo	Josh

ID	Description	Expected Result	Req(s)	Method(s)	Owner
TC-REFILL-04	Associate provider with medication	200; provider_id set	FR18	Demo	Josh
TC-REFILL-05	Inspect generated_message on a request	Contains name, dosage, user id	FR19	Demo, Insp	Josh
TC-REFILL-06	Simulate send on Pending request	200; status = Sent; sent_at set	FR20, FR22	Demo	Josh
TC-REFILL-07	Simulate send on already-Sent request	409 INVALID_STATE	FR22	Demo	Josh
TC-REFILL-08	Confirmation displayed after simulated send	UI shows confirmation	FR21	Demo	Josh
TC-REFILL-09	List refill requests for caller	200 with array, correct statuses	FR23	Demo	Josh
TC-REFILL-10	List filtered by status=Pending	Only Pending returned	FR23	Demo	Josh
TC-REFILL-11	IR5: external email handoff	Device email app opens with refill message	IR5	Demo	Josh

4.7 Performance

ID	Description	Expected Result	Req(s)	Method(s)	Owner
TC-PERF-01	Time from login response to dashboard interactive (standard seed)	≤ 2 seconds	PR1	Anal, Sim	Josh
TC-PERF-02	Time from create/edit/delete submit to confirmed save	≤ 2 seconds	PR2	Anal, Sim	Josh
TC-PERF-03	Time to compute days_remaining for one medication	≤ 1 second	PR3	Anal	Joe
TC-PERF-04	PERF-01–03 repeated against performance seed (50 accounts)	All ceilings still met	PR4	Anal, Sim	Josh

4.8 Usability

ID	Description	Expected Result	Req(s)	Method(s)	Owner
TC-USE-01	Count taps/clicks from dashboard to created medication	≤ 5	UR1	Demo, Insp	Josh
TC-USE-02	Dashboard row shows name and days_remaining together	Both visible in one view	UR2	Demo	Josh
TC-USE-03	Low-supply medication shown with color AND text	Both present	UR3	Demo, Insp	Josh
TC-USE-04	Review notification text for plain English and consistency	Pass	UR4	Insp	Josh

4.9 Interface

ID	Description	Expected Result	Req(s)	Method(s)	Owner
TC-IF-01	Render on 375-px-wide mobile viewport	Controls reachable, no horizontal scroll on primary flows	IR1	Demo	Josh
TC-IF-02	Medication entry form fields	Labeled for name, dosage, pills_remaining, pills_per_day	IR2	Demo, Insp	Josh
TC-IF-03	Dashboard lists all medications for account	Pass	IR3	Demo	Josh
TC-IF-04	Back-end exposes auth, medication, calculation endpoints conforming to openapi.yaml	Endpoints reachable and conform	IR4	Insp, Anal	Joe
TC-IF-05	Refill action launches external email client with prepopulated body	Email app opens with expected content	IR5	Demo	Josh

4.10 Database

ID	Description	Expected Result	Req(s)	Method(s)	Owner
TC-DB-01	User table: unique email and password_hash column	Schema inspection passes	DB1	Insp	Haeji
TC-DB-02	Medication table: PK and required columns	Schema inspection passes	DB2	Insp	Haeji
TC-DB-03	Medication.user_id FK enforced	Invalid user_id rejected	DB3	Insp, Anal	Haeji
TC-DB-04	pills_remaining = -1 directly via data layer	Rejected	DB4	Insp, Anal	Haeji
TC-DB-05	pills_per_day = 0 directly via data layer	Rejected	DB5	Insp, Anal	Haeji
TC-DB-06	Name length 0 or 101	Rejected	DB6	Insp, Anal	Haeji
TC-DB-07	Dosage length 0 or 21	Rejected	DB7	Insp, Anal	Haeji
TC-DB-08	Password hash — no plaintext anywhere in DB	Confirmed by direct query	DB8, SA3	Insp	Haeji
TC-DB-09	Refill status column/table constrained	Schema inspection passes	DB9	Insp	Haeji

4.11 Design Constraints and System Attributes

ID	Description	Expected Result	Req(s)	Method(s)	Owner
TC-CON-01	Dependency review for pharmacy/provider/EHR integrations	None present	DC1	Insp	Josh
TC-CON-02	Application functions with no outbound healthcare API calls	Core features work	DC2	Demo, Insp	Josh
TC-CON-03	Auth uses email+password only (no MFA, OAuth)	Confirmed by inspection of /auth endpoints	DC3	Insp	Josh
TC-SA-01	Restart application; previously stored data still present	Data persists	SA1	Demo, Sim	Josh
TC-SA-02	Send malformed but non-malicious input to each endpoint	Error returned, app stable, no data loss	SA2	Demo, Sim	Josh
TC-SA-03	Submit each create/update endpoint with missing required field	400 with field-level validation message	SA4	Demo, Insp	Josh

5. Shared procedures

5.1 Environments and test data

Environment	Purpose	Composition
Local dev (FE)	Front-end work without back-end dependency.	FE codebase + Prism mock from openapi.yaml.
Local dev (BE)	Back-end work without UI dependency.	BE service + local test DB.
Integration	Real FE calling real BE against a controlled DB.	All three tiers, seeded test database.
System / QA	End-to-end verification including mobile responsiveness and email handoff.	Built FE on mobile form factor, BE on stable build, fresh DB from seed.

Three test fixture sets (owned by Haeji, see §3.4):

- **Empty seed** — no users, no medications. For account creation and first-login tests.
- **Standard seed** — 3 users, each with 2–5 medications spanning healthy supply, low (≤ 7 days), critical (≤ 1 day), and edge values (0 pills remaining).
- **Performance seed** — 50 users with a representative spread of medication counts (matches the SRS §3.2 ceiling).

5.2 Defect handling

Defects are filed as GitHub Issues with the **defect** label. Each entry includes:

- One-line title summarizing the failure.
- Test case ID that uncovered it (if applicable).
- Affected requirement IDs.
- Environment and build identifier (commit hash).
- Reproduction steps; expected vs. actual; attached artifacts (logs, screenshots, request/response captures).
- Severity (assigned by Josh) and priority (assigned by Xander).

Severity	Definition
S1 — Critical	Blocks a primary user workflow with no workaround (e.g., login broken, data loss, security exposure). Release-blocking.
S2 — Major	Primary workflow degraded but workaround exists, or a secondary workflow blocked. Release-blocking unless explicitly accepted by Xander.
S3 — Minor	Cosmetic, edge-case, or low-frequency issue. Not release-blocking.
S4 — Trivial	Documentation, comment, or non-user-visible issue.

Lifecycle: New → Triaged → In progress → Ready for verification → Closed. Reopened if verification fails.

5.3 Traceability matrix

Maps every requirement from the SRS to the test cases that verify it. Josh keeps this current. The matrix tracks SRS §3.1 numbering; where §4.6 of the SRS uses different numbering for refill, this matrix tracks §3.1 (see §6.1).

Req	Summary	Test case(s)	Methods	Levels
FR1	Unique-email account creation	TC-AUTH-01, 02, 03	Demo, Insp	U, I, S
FR2	Password policy	TC-AUTH-04	Demo, Anal	U
FR3	Authenticate	TC-AUTH-05, 06, 07, 14, 15	Demo, Insp	I, S
FR4	Logout	TC-AUTH-08	Demo	S
FR5	Password reset	TC-AUTH-09, 10, 11, 12	Demo, Insp	I, S
FR6	Create medication	TC-MED-01–07	Demo	I, S
FR7	Store medication fields	TC-MED-01, 11	Insp, Anal	U, I
FR8	Update medication	TC-MED-11, 12	Demo, Insp	I, S
FR9	Delete medication	TC-MED-13, 14	Demo, Insp	I, S
FR10	Calculate days_remaining	TC-CALC-01–05	Anal	U
FR11	Display days_remaining	TC-CALC-06, TC-DASH-02	Demo	S
FR12	Dashboard lists medications	TC-DASH-01, 03, 04, TC-MED-08	Demo	S
FR13	Dashboard shows required fields	TC-DASH-02	Demo, Insp	S
FR14	Identify ≤7 day records	TC-MED-06, TC-NOTIF-01	Anal, Sim	U, S
FR15	Notifications at 7/3/1, anti-spam	TC-NOTIF-01–04	Anal, Sim	U, S
FR16	Display notifications	TC-NOTIF-05, 06, 07	Demo, Sim	S
FR17	Initiate refill	TC-REFILL-01, 02	Demo	S
FR18	Provider association (per §3.1)	TC-REFILL-03, 04	Demo	I, S
FR19	Generate structured request	TC-REFILL-05	Demo, Insp	I, S
FR20	Simulate send	TC-REFILL-06, 07	Demo	S
FR21	Display confirmation	TC-REFILL-08	Demo	S
FR22	Assign status	TC-REFILL-01, 06, 07	Demo, Insp	I, S
FR23	Display refill status	TC-REFILL-09, 10	Demo	S
PR1	Dashboard ≤ 2s	TC-PERF-01	Anal, Sim	S
PR2	Save ≤ 2s	TC-PERF-02	Anal, Sim	I, S
PR3	Calc ≤ 1s	TC-PERF-03	Anal	U
PR4	50-account ceiling	TC-PERF-04, TC-DASH-04	Anal, Sim	S
UR1	Create in ≤ 5 taps	TC-USE-01	Demo, Insp	S
UR2	Name + days in one view	TC-USE-02	Demo, Insp	S
UR3	Color + text low-supply	TC-USE-03, TC-NOTIF-08	Demo, Insp	S

Req	Summary	Test case(s)	Methods	Levels
UR4	Plain English notifications	TC-USE-04, TC-NOTIF-07	Insp, Demo	S
IR1	Mobile-responsive UI	TC-IF-01	Demo, Insp	S
IR2	Form labels	TC-IF-02	Demo, Insp	S
IR3	Dashboard lists medications	TC-IF-03	Demo, Insp	S
IR4	BE services exposed	TC-IF-04	Insp, Anal	I
IR5	External email handoff	TC-IF-05, TC-REFILL-11	Demo, Insp	I, S
DB1	User entity	TC-DB-01	Insp, Anal	U, I
DB2	Medication entity	TC-DB-02	Insp	U, I
DB3	User–Medication 1:N	TC-DB-03, TC-MED-10, 14, 15	Insp, Anal	U, I
DB4	pills_remaining ≥ 0	TC-MED-05, TC-DB-04	Insp, Anal	U
DB5	pills_per_day ≥ 1	TC-MED-07, TC-DB-05	Insp, Anal	U
DB6	Name 1–100 chars	TC-MED-02, 03, TC-DB-06	Insp, Anal	U
DB7	Dosage 1–20 chars	TC-MED-04, TC-DB-07	Insp, Anal	U
DB8	Password complexity	TC-AUTH-04, TC-DB-08	Insp, Anal	U
DB9	Refill status persisted	TC-DB-09, TC-REFILL-01, 06	Insp	U, I
DC1	No pharmacy/EHR integration	TC-CON-01	Insp	S
DC2	No real-time healthcare data	TC-CON-02	Insp	S
DC3	Single-factor auth	TC-CON-03, TC-AUTH-14	Insp, Demo	S
SA1	Preserve data	TC-SA-01	Demo, Sim	U, S
SA2	Stable on bad input	TC-SA-02, TC-DASH-03	Demo, Sim	U, S
SA3	Password hashed	TC-AUTH-13, TC-DB-08	Insp, Anal	U, I
SA4	Validate before storing	TC-SA-03, all 400-response cases	Insp, Demo	U, I

5.4 Release exit criteria

Josh confirms all of the following before a release is approved:

- Every requirement in §5.3 has at least one executed test case with a pass result.
- No open S1 defects.
- No more than 2 open S2 defects, each with an accepted workaround documented.
- The regression suite passes end-to-end on the system environment.
- Performance ceilings PR1–PR4 are measured and met.

5.5 Testing milestones

Detailed scheduling is owned by Xander (Scrum Master) per sprint. The milestones below are the testing-side gates the plan is built around.

Milestone	Definition of done
Test plan approved	This document reviewed and accepted by the team.
Test fixtures available	Empty, standard, and performance seeds in the repo.
FE unit tests in place	Framework running in CI; coverage for AuthController and AuthService.
BE unit tests in place	Framework running in CI; coverage for validators, password hashing, supply calculator, notification engine.
API contract tests passing against Prism	Every endpoint case in §4 runs against the mock without spec violations.
Integration milestone	FE pointed at real BE successfully completes auth and medication CRUD.
System milestone	Full §4 suite executed; results in the traceability matrix.
Release candidate	Exit criteria met (§5.4).

6. Notes

6.1 Open SRS discrepancy

The verification mapping in SRS §4.6 was written against an earlier version of §3.1. Specifically:

- §4.6 labels FR18 as "Copy refill request message" and FR19 as "Open external email/share interface." §3.1 defines FR18 as provider association and FR19 as structured-request generation.
- §4.6 stops at FR20 and does not cover FR21 (confirmation), FR22 (status assignment), or FR23 (status display).

This plan tracks SRS §3.1. The discrepancy should be resolved in the next SRS revision; this plan will be updated to match. Xander to call this out at the next planning meeting.

6.2 Risks

Workload concentration on Xander. With Parker gone, Xander is now PO, Scrum Master, and Front-End Tech Lead simultaneously. Front-end testing competes with planning and acceptance for the same person's time. If FE testing slips, the integration milestone slips. Consider scoping back FE unit-test depth (focus on AuthController and form validators first) and leaning more heavily on Prism-driven integration tests, which Xander can run quickly.

Risk	Impact	Mitigation
Back-end implementation lags front-end.	Delayed system testing.	FE tests run against Prism in the interim. Integration suite ready to run when BE is up.
Email service for password reset is non-trivial to stand up.	FR5 hard to demo end-to-end.	Use logged-to-disk or captured-email mock; defer real SMTP.
Mobile responsiveness needs device variety we may lack.	Incomplete IR1 verification.	Use browser device emulation; flag device-specific behavior as a known limitation.
Notification anti-spam (FR15) is the most subtle logic in the system.	Latent defects.	Heavy unit-test coverage with simulated time-series of supply changes (TC-NOTIF-01-04).
Spec drift between openapi.yaml and back-end.	Integration breakage at convergence.	Contract tests run against Prism and against real BE; any divergence is a defect.
Performance ceilings (PR1-PR3) sensitive to environment.	False pass/fail.	Run performance cases on a designated environment, not on shared CI.

6.3 Assumptions

- The team adopts one unit-test framework per tier; the choice is left to the tech leads.
- CI is available for running unit and integration suites on every PR.
- openapi.yaml remains the authoritative API contract throughout the project.
- Test fixtures live in the repo and are versioned with the code.

6.4 Approvals

This plan takes effect once the team members below have reviewed and accepted it.

Role	Name	Signature	Date
PO / SM / FE Lead	Xander Weibel		
Back-End Lead	Joe Tolley		
Database Lead	Haeji Na		
QA / CM / DM	Josh Palmer		